Discrete Structure

Unit 3. Automata Theory

3.1 Finite State Automata:

A **Finite Automata** (FA), also known as a **Finite State Machine (FSM)**, is an abstract mathematical model used to recognize patterns or formal languages. It is the simplest model of computation, consisting of a finite set of states and rules for transitioning between them based on input symbols.

A finite automaton can be defined as a 5-tuple:

M = { Q, Σ, q, F, δ }, where:

- Q: Finite set of states
- Σ: Set of input symbols
- q: Initial state
- F: Set of final states
- δ: Transition function can be express δ:Q×Σ→Q

Types of Finite Automata

There are two types of finite automata:

- Deterministic Finite Automata (DFA)
- Non-Deterministic Finite Automata (NFA)

1. Deterministic Finite Automata (DFA)

In a DFA, **for each state and each input symbol, there is exactly one transition** to a next state. **Null (ε) transitions are not allowed**, and a transition must be defined for **every input symbol in every state**.

DFA consists of 5 tuples {Q, Σ, q, F, δ}.
Q : set of all states.
Σ : set of input symbols. (Symbols which machine takes as input)
q : Initial state. (Starting state of a machine)
F : set of final state.
δ : Transition Function, defined as δ : Q X Σ --> Q.

Example:

Construct a DFA that accepts all strings ending with 'a'.

Given:

Σ = {a, b},

Q = {q0, q1},

F = {q1}

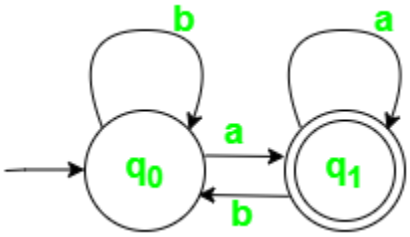


Fig 1. State Transition Diagram for DFA with Σ = {a, b}

State \ Symbol	a	b
q_0	q_1	q_0
q_1	q_1	q_0

In this example, if the string ends in 'a', the machine reaches state q1, which is an accepting state.

2) **Non-Deterministic Finite Automaton (NFA)**,

In a **Non-Deterministic Finite Automaton (NFA)**, for a given input symbol, the machine can move to **one or more states**, or even **no state**. Therefore, the next state is **not uniquely determined**, which is why it is called **non-deterministic**.

Formal Definition

An NFA is represented by a 5-tuple

$$(Q, \Sigma, \delta, q_0, F)$$

where:

- Q= finite set of states
- Σ= finite input alphabet

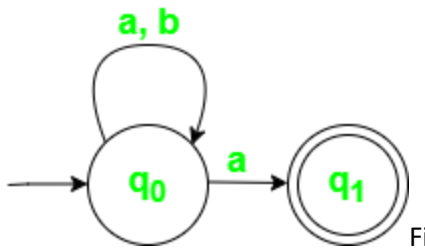
- $\delta: Q \times (\Sigma \cup \{\varepsilon\}) \rightarrow 2^Q$
  *(Transitions may include **ε-moves**, and may lead to multiple states)*
- $q_0 \in Q$= initial state
- $F \subseteq Q$= set of final states
- Construct an NFA that accepts strings ending in 'a'.
- Given:
- ∑ = {a, b},
- Q = {q0, q1},
- F = {q1}
- 

Fig 2. State Transition Diagram for NFA with Σ = {a, b}
- State Transition Table for above Automaton,

State \ Symbol	a	b
q_0	$\{q_0, q_1\}$	q_0
q_1	φ	φ

- In an NFA, if any transition leads to an accepting state, the string is accepted.

- **NFA vs DFA**

DFA	NFA
Deterministic: exactly one transition per input symbol	Non-deterministic: multiple transitions per input symbol allowed
No ε (null) moves	Allows ε (null) moves
Simpler but sometimes larger in design	More flexible and easier to design
Next state is uniquely determined	Next state may be multiple possibilities
Recognizes regular languages; NFAs can be converted to DFA	Recognizes regular languages; equivalent in power to DFA

3.1.3 Regular Expression (RE)

A **Regular Expression** is a **formal way to describe a set of strings** over a given alphabet.
It is mainly used to **define patterns** and is closely related to **finite automata**.

In theory of computation, regular expressions are used to **represent regular languages**.

Formal Definition of Regular Expression

Let **Σ (Sigma)** be an alphabet (a finite set of symbols).

A regular expression over Σ is defined **recursively** as follows:

1. **Ø (empty set)** is a regular expression
   → It represents **no string**
2. **ε (epsilon)** is a regular expression
   → It represents the **empty string**

3. **a ∈ Σ** (any symbol from alphabet) is a regular expression
   → It represents the string **"a"**
4. If $\mathbf{R_1}$ and $\mathbf{R_2}$ are regular expressions, then:
   - $\mathbf{(R_1 + R_2)}$ → Union (OR)
   - $\mathbf{(R_1 \cdot R_2)}$ → Concatenation
   - $\mathbf{(R_1)^*}$ → Kleene Star (zero or more repetitions)

   are also regular expressions.

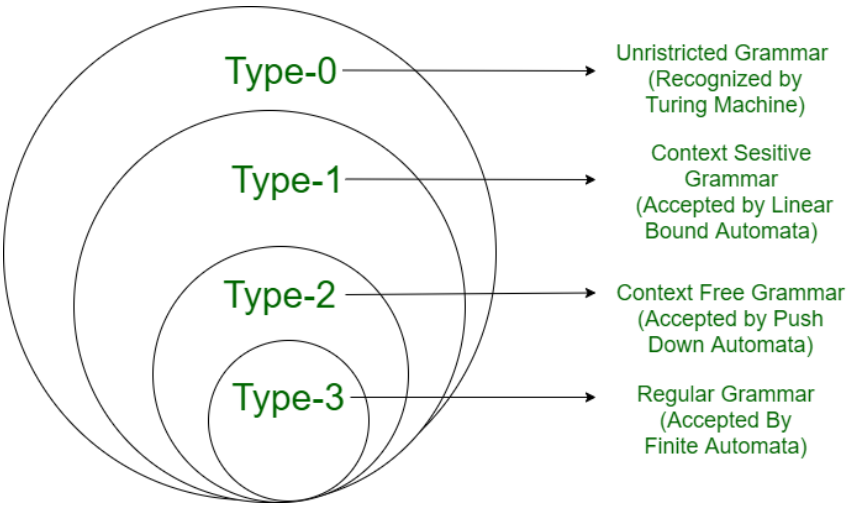
3.2 Grammer Concept

A **Grammar** is a **formal way to define a language** using a set of rules.
It describes **how strings are generated** from symbols.

3.2.1 Chomsky Hierarchy:

The Chomsky Hierarchy is a classification of formal languages into four types based on the restrictions of their grammars and the computational power required to recognize them. It covers all language types, from regular languages (Type 3) with the simplest grammar to recursively enumerable languages (Type 0) with the most complex grammar.

According to Chomsky hierarchy, grammar is divided into 4 types as follows:



Types of Chomsky Hierarchy

Type-0: Unrestricted Grammar

- No restriction on production rules (α → β).
- Generates **recursively enumerable languages**.
- Recognized by **Turing Machine**.

Type-1: Context-Sensitive Grammar (CSG)

- Productions depend on context (αAβ → αγβ).
- Length of RHS ≥ LHS.
- Recognized by **Linear Bounded Automaton**.

Type-2: Context-Free Grammar (CFG)

- Single non-terminal on LHS (A → γ).
- Used in **programming languages**.
- Recognized by **Pushdown Automaton**.

Type-3: Regular Grammar

- Linear productions (A → aB or A → a).
- Generates **regular languages**.
- Recognized by **Finite Automata**.

3.2.2 Context free grammar: Derivation, Parse TREE, Language computation and Grammar design:

A **Context Free Grammar (CFG)** is a grammar in which **each production rule has exactly one non-terminal on the left-hand side**.

Formal Definition

A **Context-Free Grammar** is a 4-tuple:
G = (V, Σ, P, S) where:

- **V**: Set of non-terminals (variables)
- **Σ**: Set of terminals (alphabet)
- **P**: Set of production rules (A → α, where A ∈ V, α ∈ (V ∪ Σ)*)
- **S**: Start symbol (S ∈ V)

(a) Derivation

Derivation is the process of generating a string from the start symbol by repeatedly applying production rules until only terminal symbols remain.

If a string w can be derived from start symbol S, it is written as:

$S \Rightarrow^* w$

Types of Derivation

Leftmost Derivation

In **leftmost derivation**, the leftmost non-terminal is replaced at each step.

Example

Grammar:

$S \rightarrow aSb \mid \varepsilon$

Derivation of aabb:

S

$\Rightarrow aSb$

$\Rightarrow aaSbb$

$\Rightarrow aabb$

Rightmost Derivation

In **rightmost derivation**, the rightmost non-terminal is replaced at each step.

Example

Grammar:

$S \rightarrow aSb \mid \varepsilon$

Derivation of aabb:

S

$\Rightarrow aSb$

$\Rightarrow aSbb$

$\Rightarrow aabb$

Parse Tree

A parse tree, also known as derivation tree is a graphical representation of the derivation of a string according to the production rules of a CFG. It shows how a start symbol of a CFG can be transformed into a terminal string, using applying production rules in a sequence.

Properties of Parse Tree

- The root node represents the start symbol.
- Internal nodes represent non-terminal symbols.
- Leaf nodes represent terminal symbols or ε.
- The left-to-right order of leaf nodes gives the derived string.

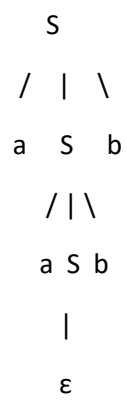
Parse Tree Example 1:

Grammar:

S → aSb

S → ε

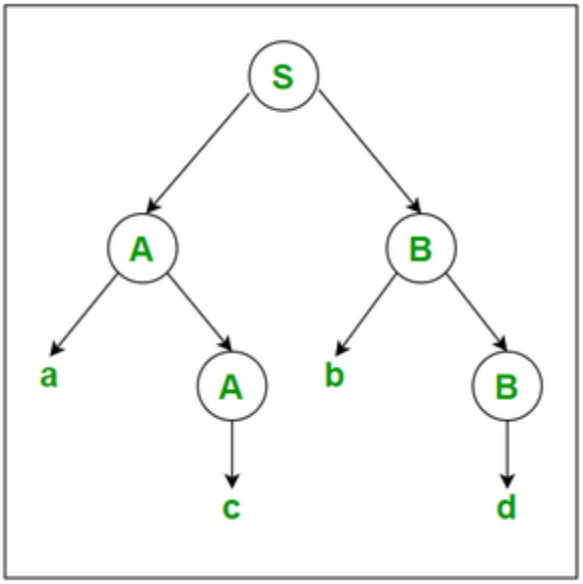
String: aabb



Example-2: Let us take another example of Grammar (Production Rules).

S -> AB
A -> c/aA
B -> d/bB

The input string is "acbd", then the Parse Tree is as follows:



(c) Language Computation

Language computation in CFG refers to the process of defining and generating the **set of all valid strings** that can be produced using the grammar's production rules starting from the start symbol.

Formally, for a CFG **G = (V, T, P, S)**, the language generated by G is:

$L(G) = \{ w \in T^* \mid S \Rightarrow^* w \}$

Where:

- **T*** → Set of all strings made of terminal symbols
- **⇒*** → Zero or more derivation steps
- **w** → A valid string of the language

How CFG Computes a Language

CFG computes a language by:

1. Starting from the **start symbol (S)**
2. Repeatedly applying **production rules**
3. Replacing **non-terminals** with terminals
4. Continuing until **only terminals remain**

If a string can be derived this way, it belongs to the language.

Example 1: Simple Language

Grammar:

$S \rightarrow aS \mid b$

Derivations:

$S \Rightarrow b$

$S \Rightarrow aS \Rightarrow ab$

$S \Rightarrow aS \Rightarrow aS \Rightarrow aab$

Language Generated:

$L(G) = \{ a^n b \mid n \geq 0 \}$

This language includes:

b, ab, aab, aaab, ...

(d) Grammar Design

Grammar design is the process of creating a CFG that **correctly and efficiently represents the syntax of a language**.

A well-designed grammar:

- Generates **all valid strings**
- Rejects **invalid strings**
- Is **clear, structured, and unambiguous**

Example 1: Arithmetic Expression Grammar

Poor (Ambiguous) Grammar:

$E \rightarrow E + E \mid E * E \mid id$

Problem:

- Expression id + id * id has **multiple parse trees**
- Operator precedence is unclear

Improved (Unambiguous) Grammar:

E → E + T | T

T → T * F | F

F → (E) | id

Advantages:

- * has higher precedence than +
- Left associativity is maintained
- Suitable for compiler parsing

3.2.3. Regular grammar to finite Automata and vice versa (Students part)